

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи № 1
« Організація професійного Python-проєкту та Git »

Виконала:
студентка 3 курсу
групи ФЕП-31
Антошкова Аліна Олександрівна
Перевірив:
Доц. Рабош Роман Васильович

Мета: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Варіант 12

Постановка завдання

Розробити консольний застосунок «Фінансовий трекер». Для кожної операції зберігати: дату, категорію, суму та тип (дохід/витрата). Реалізувати додавання операції, підрахунок доходів, підрахунок витрат, розрахунок балансу та групування витрат за категоріями. Забезпечити src-layout, type hints, меню та валідацію вхідних значень.

Хід роботи:

1. Створила новий репозиторій для лабораторних

```
Alina@ASUS MINGW64 ~/Desktop/пайтон (main)
$ git clone "https://github.com/AntoskovaA/paython.git"
Cloning into 'paython'...
warning: You appear to have cloned an empty repository.

Alina@ASUS MINGW64 ~/Desktop/пайтон (main)
$ cd paython

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

2. Додавання файлів базової конфігурації проєкту (.gitignore, pyproject.toml) та виконання початкового коміту

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git status
On branch main

No commits yet

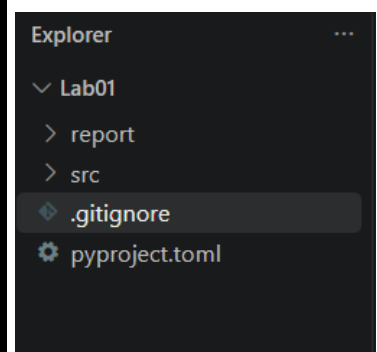
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    Lab1/

nothing added to commit but untracked files present (use "git add" to track)

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git commit -m "initial commit, setup project"
[main (root-commit) 2215b9f] initial commit, setup project
 2 files changed, 42 insertions(+)
 create mode 100644 Lab1/.gitignore
 create mode 100644 Lab1/pyproject.toml

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 804 bytes | 804.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AntoskovaA/paython.git
 * [new branch]      main -> main
```



3. Налаштування віртуального оточення та створення структури src-layout

```

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git add .

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore
    new file:   pyproject.toml
    new file:   src/finance_tracker/__init__.py
    new file:   tests/__init__.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git commit -m "created folders for project, init files and virtual environment"
[main ecb0c51] created folders for project, init files and virtual environment
4 files changed, 45 insertions(+)
create mode 100644 Lab01/.gitignore
create mode 100644 Lab01/pyproject.toml
create mode 100644 Lab01/src/finance_tracker/__init__.py
create mode 100644 Lab01/tests/__init__.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (10/10), 1.16 KiB | 1.16 MiB/s, done.
Total 10 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AntoskovaA/paython.git
 2215b9f..ecb0c51  main -> main

```

```

• PS C:\Users\Alina\Desktop\пайтон\python\Lab01> python -m venv .venv
• PS C:\Users\Alina\Desktop\пайтон\python\Lab01> .venv\Scripts\activate
(.venv) PS C:\Users\Alina\Desktop\пайтон\python\Lab01> New-Item -Path
tests\__init__.py -ItemType File -Force

```

```

Directory: C:\Users\Alina\Desktop\пайтон\python\Lab01\tests

Mode                LastWriteTime         Length Name
----                -
-a----           18.09.2026   13:34              0 __init__.py

```

4. Створення модулів models.py та services.py, фіксація коміту та push на GitHub

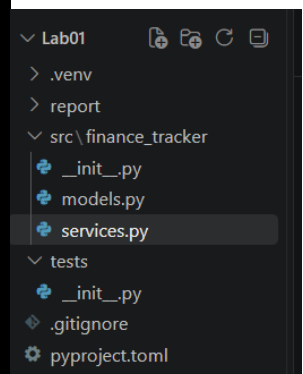
```

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git add src/finance_tracker/models.py src/finance_tracker/services.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git commit -m "Add transaction model and calculation services"
[main 74be2b5] Add transaction model and calculation services
4 files changed, 60 insertions(+), 42 deletions(-)
create mode 100644 Lab01/src/finance_tracker/models.py
create mode 100644 Lab01/src/finance_tracker/services.py
delete mode 100644 Lab01/.gitignore
delete mode 100644 Lab01/pyproject.toml

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 1.17 KiB | 1.17 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/AntoskovaA/paython.git
  ecb0c51..74be2b5  main -> main

```



5. Створення точки входу програми та документації

```
Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git add src/finance_tracker/main.py README.md

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git commit -m "Add main entry point and README"
[main 41d948a] Add main entry point and README
2 files changed, 111 insertions(+)
create mode 100644 Lab01/README.md
create mode 100644 Lab01/src/finance_tracker/main.py

Alina@ASUS MINGW64 ~/Desktop/пайтон/paython/Lab01 (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 2.09 KiB | 1.05 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AntoskovaA/paython.git
74be2b5..41d948a main -> main
```

6. Встановлення пакета в режимі розробки та демонстрація роботи застосунку

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\python\Lab01> python -m pip install -e .
Obtaining file:///C:/Users/Alina/Desktop/%D0%BF%D0%B0%D0%B9%D1%82%D0%BE%D0%BD/paython/Lab01
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: finance-tracker
Building editable for finance-tracker (pyproject.toml) ... done
Created wheel for finance-tracker: filename=finance_tracker-0.1.0-0.editable-py3-none-any.whl size
=1846 sha256=daa0f1782379267934d6cbaf3beeceb29f9fb7cb8f30200c3ca7eafe07651beb
Stored in directory: C:\Users\Alina\AppData\Local\Temp\pip-ephem-wheel-cache-_qebw76d\wheels\73\c9
\5e\cdfaf2e69bd748ffb694e771286e903e136700609652a5e94f
Successfully built finance-tracker
Installing collected packages: finance-tracker
Successfully installed finance-tracker-0.1.0

[notice] A new release of pip is available: 24.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
(.venv) PS C:\Users\Alina\Desktop\пайтон\python\Lab01> python -m finance_tracker.main

=== Меню Фінансового трекера ===
1. Всі операції
2. Додати операцію
3. Баланс і статистика
4. Вихід
Ваш вибір (1-4): 1

--- Список операцій ---
Дата      | Тип      | Категорія      | Сума
-----|-----|-----|-----
2026-09-01 | income   | Зарплата       | 28000.00 грн
2026-09-02 | expense  | Продукти       | 1450.00 грн
2026-09-05 | expense  | Транспорт      | 350.00 грн
2026-09-10 | income   | Фриланс        | 5000.00 грн
2026-09-12 | expense  | Продукти       | 920.00 грн
```

```

=== Меню Фінансового трекера ===
1. Всі операції
2. Додати операцію
3. Баланс і статистика
4. Вихід
Ваш вибір (1-4): 3

--- Фінансова статистика ---
Доходи : 33000.00 грн
Витрати: 2720.00 грн
Баланс : 30280.00 грн

Витрати за категоріями:
* Продукти: 2370.00 грн
* Транспорт: 350.00 грн

=== Меню Фінансового трекера ===
1. Всі операції
2. Додати операцію
3. Баланс і статистика
4. Вихід
Ваш вибір (1-4): 4
Вихід з програми.

```

7. Верифікація стану Git-репозиторію

```

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab01 (main)
$ git log --oneline
41d948a (HEAD -> main, origin/main) Add main entry point and README
74be2b5 Add transaction model and calculation services
ecb0c51 created folders for project, init files and virtual environment
2215b9f initial commit, setup project

Alina@ASUS MINGW64 ~/Desktop/пайтон/python/Lab01 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```

Опис створених модулів

models.py — модуль визначення предметної області та структур даних. Містить датаклас Transaction для представлення фінансової операції (дата, категорія, сума, тип) та користувацький клас винятку ValidationError для валідації вхідних значень у методі `__post_init__`.

services.py — модуль бізнес-логіки застосунку. Складається з чистих функцій для додавання операцій, підрахунку загальних доходів і витрат, визначення чистого балансу та групування витрат за окремими категоріями. Не містить операцій введення/виведення, що забезпечує незалежність логіки від консолі.

main.py — точка входу до програми (application entry point). Реалізує консольний інтерфейс користувача: формування демонстраційного набору даних, табличне форматування операцій, діалогове додавання нових записів із валідацією та виведення фінансової аналітики. Запуск захищено конструкцією `if __name__ == "__main__":`.

Висновок:

Під час виконання лабораторної роботи № 1 було опановано фундаментальні принципи побудови сучасного Python-проєкту та культуру контролю версій Git. Створено ізольоване середовище розробки за допомогою модуля `venv`, реалізовано стандартну структуру проєкту за архітектурним шаблоном `src-layout`, що запобігає некоректному імпорту локальних файлів без інсталяції, та налаштовано метадані й параметри збірки у файлі `pyproject.toml`.

У межах індивідуального завдання (Варіант 12 — «Фінансовий трекер») розроблено модульний консольний застосунок із чітким розділенням моделей даних (`models.py`), бізнес-логіки обчислення доходів, витрат і балансу (`services.py`) та інтерфейсу користувача (`main.py`). Для сутності транзакції застосовано декоратор `dataclass`, реалізовано сувору типізацію (`type hints`) та валідацію коректності числових і рядкових значень через власний клас винятків `ValidationError`. Локальний пакет успішно встановлено в режимі редагування (`editable mode`), а всі етапи розробки зафіксовано в системі Git через логічні коміти та синхронізовано з віддаленим репозиторієм на GitHub.

Додаток:

1. Файл src/finance_tracker/models.py

```
from dataclasses import dataclass
```

```
from datetime import date
```

```
class ValidationError(Exception):
```

```
    """Помилка валідації вхідних даних."""
```

```
    pass
```

```
@dataclass
```

```
class Transaction:
```

```
    op_date: date
```

```
    category: str
```

```
    amount: float
```

```
    op_type: str # "income" або "expense"
```

```
    def __post_init__(self) -> None:
```

```
        if self.amount <= 0:
```

```
            raise ValidationError("Сума повинна бути більшою за нуль!")
```

```
        if self.op_type not in ("income", "expense"):
```

```
            raise ValidationError("Тип має бути тільки income або expense!")
```

```
        if not self.category.strip():
```

```
            raise ValidationError("Категорія не може бути порожньою!")
```

```
        self.category = self.category.strip()
```

2. Файл src/finance_tracker/services.py

```
from finance_tracker.models import Transaction
```

```
def add_transaction(data_list: list[Transaction], item: Transaction) -> None:
    data_list.append(item)
```

```
def get_total_income(data_list: list[Transaction]) -> float:
    total = 0.0
    for t in data_list:
        if t.op_type == "income":
            total += t.amount
    return total
```

```
def get_total_expense(data_list: list[Transaction]) -> float:
    total = 0.0
    for t in data_list:
        if t.op_type == "expense":
            total += t.amount
    return total
```

```
def get_balance(data_list: list[Transaction]) -> float:
    return get_total_income(data_list) - get_total_expense(data_list)
```

```
def get_category_expenses(data_list: list[Transaction]) -> dict[str, float]:
    result = {}
    for t in data_list:
        if t.op_type == "expense":
            if t.category in result:
```

```

        result[t.category] += t.amount
    else:
        result[t.category] = t.amount
return result

```

3. Файл src/finance_tracker/main.py

```

from datetime import date, datetime
from finance_tracker.models import Transaction, ValidationError
from finance_tracker.services import (
    add_transaction,
    get_balance,
    get_category_expenses,
    get_total_expense,
    get_total_income,
)

```

```

def demo_data() -> list[Transaction]:
    return [
        Transaction(date(2026, 9, 1), "Зарплата", 28000.0, "income"),
        Transaction(date(2026, 9, 2), "Продукти", 1450.0, "expense"),
        Transaction(date(2026, 9, 5), "Транспорт", 350.0, "expense"),
        Transaction(date(2026, 9, 10), "Фриланс", 5000.0, "income"),
        Transaction(date(2026, 9, 12), "Продукти", 920.0, "expense"),
    ]

```

```

def show_transactions(items: list[Transaction]) -> None:
    if not items:
        print("\nСписок операцій порожній.")

```

```

    return

print("\n--- Список операцій ---")
print(f'{'Дата':12} | {'Тип':8} | {'Категорія':15} | {'Сума':10}')
print("-" * 52)
for t in items:
    print(f'{'str(t.op_date):12} | {'t.op_type':8} | {'t.category':15} | {'t.amount:10.2f}'
грн")

def add_new_transaction(items: list[Transaction]) -> None:
    print("\n--- Нова операція ---")
    raw_date = input("Дата (РРРР-ММ-ДД, Enter - сьогодні): ").strip()
    try:
        t_date = datetime.strptime(raw_date, "%Y-%m-%d").date() if raw_date else
date.today()
    except ValueError:
        print("Помилка: невірний формат дати!")
        return

    t_type_num = input("Тип (1 - income, 2 - expense): ").strip()
    if t_type_num == "1":
        t_type = "income"
    elif t_type_num == "2":
        t_type = "expense"
    else:
        print("Помилка: оберіть 1 або 2!")
        return

    cat = input("Категорія: ").strip()

```

try:

```
amt = float(input("Сума: ").strip())
new_tx = Transaction(t_date, cat, amt, t_type)
add_transaction(items, new_tx)
print("Транзакцію успішно додано!")
```

except ValueError:

```
print("Помилка: сума має бути числом!")
```

except ValidationError as err:

```
print(f"Помилка валідації: {err}")
```

def show_statistics(items: list[Transaction]) -> None:

```
inc = get_total_income(items)
exp = get_total_expense(items)
bal = get_balance(items)
cat_exp = get_category_expenses(items)
```

```
print("\n--- Фінансова статистика ---")
```

```
print(f"Доходи : {inc:.2f} грн")
```

```
print(f"Витрати: {exp:.2f} грн")
```

```
print(f"Баланс : {bal:.2f} грн")
```

```
print("\nВитрати за категоріями:")
```

```
for cat, val in cat_exp.items():
```

```
    print(f"    * {cat}: {val:.2f} грн")
```

def main() -> None:

```
tx_list = demo_data()
```

while True:

```
print("\n=== Меню Фінансового трекера ===")
print("1. Всі операції")
print("2. Додати операцію")
print("3. Баланс і статистика")
print("4. Вихід")
```

```
choice = input("Ваш вибір (1-4): ").strip()
if choice == "1":
    show_transactions(tx_list)
elif choice == "2":
    add_new_transaction(tx_list)
elif choice == "3":
    show_statistics(tx_list)
elif choice == "4":
    print("Вихід з програми.")
    break
else:
    print("Невідома команда!")
```

```
if __name__ == "__main__":
    main()
```

4. Конфігураційний файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "finance-tracker"
```

```
version = "0.1.0"
```

```
description = "Financial Tracker console application"
```

```
requires-python = ">=3.11"
```

```
[project.scripts]
```

```
finance-tracker = "finance_tracker.main:main"
```

5. Файл документації README.md

```
# Financial Tracker (Lab 01)
```

Консольний застосунок для обліку особистих фінансів (Варіант 12).

```
## Встановлення
```

```
```bash
```

```
python -m pip install -e .
```

## Відповіді на контрольні запитання

### 1. Що таке Python-проект?

Це не просто один файл із кодом, а структурована робоча папка, де є вихідний код, поділений на файли, налаштування середовища, конфігураційні файли збірки, документація та тести.

### 2. Чим проєкт відрізняється від одного Python script?

Скрипт — це один автономний файл для швидкої задачі, який запускають напряму. Проєкт має чітку структуру, власні модулі, керування залежностями через конфіг-файли та може встановлюватися як пакет.

### 3. Що таке Python module?

Модуль — це будь-який звичайний файл із розширенням .py, код з якого (функції чи класи) можна імпортувати в інші файли.

### 4. Що таке Python package?

Пакет — це папка, всередині якої лежать пов'язані між собою .py модулі та обов'язковий файл \_\_init\_\_.py.

### 5. Для чого потрібний \_\_init\_\_.py?

Він показує інтерпретатору Python, що цю папку потрібно сприймати саме як пакет для імпорту, а також виконує стартовий код під час завантаження пакета.

## **6. Що таке src-layout?**

Це спосіб оформлення проєкту, коли весь код програми ховають в окрему директорію `src/`, а не тримають у корені.

## **7. Які переваги має src-layout?**

Він гарантує, що програма працює саме як встановлений пакет, не дає випадково імпортувати сирі локальні файли в обхід збірки та тримає корінь репозиторію чистим від кешу `__pycache__`.

## **8. Для чого використовується virtual environment?**

Щоб ізолювати встановлені бібліотеки одного проєкту від іншого та від глобального системного Python, уникаючи конфліктів версій.

## **9. Як створити .venv?**

Командою в терміналі: `python -m venv .venv`.

## **10. Як активувати virtual environment?**

На Windows: `.venv\Scripts\activate`, на Linux чи macOS: `source .venv/bin/activate`.

## **11. Для чого використовується pip?**

Це менеджер пакетів для завантаження, встановлення, оновлення та видалення сторонніх бібліотек з офіційного репозиторію PyPI.

## **12. Чому рекомендується python -m pip?**

Цей запис запускає `pip` саме від того конкретного інтерпретатора Python, який зараз активний у системі або віртуальному середовищі, що запобігає встановленню бібліотек не туди.

## **13. Для чого потрібний pyproject.toml?**

Це уніфікований головний файл конфігурації сучасного Python-проєкту, де прописуються інструменти збірки, залежності, версія проєкту та налаштовуються консольні команди.

## **14. Що містить секція [project]?**

Основні дані про проєкт: його назву, версію, короткий опис, мінімальну версію Python, авторів та список залежностей.

## **15. Що таке editable installation?**

Встановлення пакета командою `pip install -e .`, коли код прив'язується через посилання: будь-які правки у вихідних файлах одразу видно під час запуску без повторної переінсталяції.

## **16.Для чого використовується `if __name__ == "__main__":`?**

Щоб відрізнити прямий запуск файлу від його імпорту: код усередині цієї умови виконується тільки тоді, коли файл запущено користувачем напряму, а не підключено як бібліотеку.

## **17.Що таке Git?**

Це система контролю версій, яка відстежує історію змін у файлах і дозволяє легко повертатися до старих станів чи працювати паралельно.

## **18.Що таке repository?**

База даних Git (папка `.git`), у якій зберігається вся історія коммітів, налаштувань, міток та гілок проєкту.

## **19.Що таке commit?**

Це збережений «знімок» змін у файлах із власним повідомленням автора та унікальним хешем.

## **20.Для чого використовується `git status`?**

Щоб перевірити стан файлів у проєкті: які файли було змінено, які підготовлені до збереження, а які ще взагалі не відстежуються.

## **21.Для чого `git add`?**

Щоб підготувати (проіндексувати) додані чи змінені файли перед створенням наступного коміту.

## **22.Для чого `git commit`?**

Щоб остаточно записати підготовлені файли як новий пункт в історії змін репозиторію.

## **23.Що показує `git log`?**

Список усіх створених комітів у гілці по порядку: хеш, автора, дату та опис.

## **24.Для чого потрібен `.gitignore`?**

Щоб Git автоматично ігнорував непотрібні файли (кеш, віртуальне середовище, системні логи) і не тягнув їх у репозиторій.

## **25.Чому `.venv` не потрібно комітити?**

Там дуже багато важких файлів, прив'язаних до конкретної операційної системи; інший розробник завжди може створити `.venv` заново однією командою.

## **26.Для чого потрібний README?**

Це головний посібник проєкту: він пояснює, для чого створена програма, які до неї вимоги, як її встановити та як запустити.

### **27.Що таке type hints?**

Вказування типів змінних та параметрів функцій (наприклад, `: int` чи `-> str`), що спрощує читання коду, допомагає редактору підказувати помилки і робить програму надійнішою.

### **28.Для чого програму розділяють на modules?**

Щоб не писати все в одному довгому файлі, де важко розібратися: коли кожен файл відповідає за щось одне, код значно простіше читати, виправляти та тестувати.

### **29.Що таке business logic?**

Це безпосередня математика та алгоритми розв'язку задачі (наприклад, підрахунок доходів, балансу чи знижок), відокремлені від інтерфейсу чи візуалу.

### **30.Чому потрібно відокремлювати application entry point від business logic?**

Щоб логіка обчислень була повністю незалежною: ті самі розрахункові функції можна було викликати з консолі, з веб-сайту чи з автоматичних тестів, не чіпаючи самі математичні формули.